

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ БОГДАНА ХМЕЛЬНИЦЬКОГО

Факультет обчислювальної техніки, інтелектуальних та управляючих систем
Кафедра інформаційних технологій

ЗВІТ

з дисципліни: Проектування та архітектура ПЗ

з лабораторної роботи №4

**Тема: Реалізація спроектованої системи за допомогою Layered
Architecture pattern**

Виконав:

студент групи КС-23 Тукало Іван Валерійович

Перевірив:

Кандидат технічних наук Мисник Богдан Вікторович

Завдання:

Реалізувати 5 сценаріїв до застосунку, який ви розробляєте з першої лабораторної роботи. Це має бути веб застосунок з проєктованим вами REST API (2-га лабораторна робота) та структурований на основі layered architecture pattern. У вибраних сценаріях необхідно імплементувати визначені вами 3-4 атрибути якості (3-тя лабораторна робота). В звіті потрібно розгорнуто описати які атрибути були вибрані та яким чином реалізовані в застосунку.

Приклад знаходить в [github](https://github.com/nvakulenko/architecture-lab4/) репозиторії <https://github.com/nvakulenko/architecture-lab4/>

Приклад написаний на мові Java. Якщо ви пишете на іншій мові пропоную організувати ваш код згідно цього ж патерну але вашою мовою, якщо ваш фреймворк пропонує інший архітектурний патерн, використайте його, розкажіть при захисті про цю структуру та реалізуйте завдання згідно неї.

Корисні посилання

<https://kamilmazurek.pl/layered-architecture-template>

<https://priyalwalpita.medium.com/software-architecture-patterns-layered-architecture-a3b89b71a057>

<https://medium.com/@ahmetdede/java-spring-layered-architecture-605fb13198eb>

Варіант 2 - Автоматизація роботи бібліотекаря.

Порядок виконання:

Застосунок реалізовано за допомогою багаторівневої архітектури (Layered Architecture) на базі фреймворку ASP.NET Core мовою C#. Цей підхід дозволив чітко відокремити логіку обробки мережових запитів від бізнес-правил та механізмів роботи з базою даних. Рівень представлення (Presentation Layer) побудовано за допомогою REST-контролерів, які приймають HTTP-запити, проводять початкову валідацію вхідних об'єктів передачі даних (DTO) та повертають відповідні статуси у форматі JSON. Сервісний рівень (Application Layer) інкапсулює основну бізнес-логіку системи, таку як перевірка доступності примірників книг, створення ордерів на видачу літератури та реєстрація читачів. Рівень даних (Data Layer) відповідає за безпосередню взаємодію з базою даних через патерн Repository та систему об'єктно-реляційного відображення Entity Framework Core з використанням In-Memory бази даних для забезпечення високої швидкодії під час розробки та тестування.

Під час розробки було імплементовано чотири ключові атрибути якості. Перший атрибут продуктивність (Performance) реалізовано завдяки використанню асинхронних операцій (async та await) на всіх архітектурних рівнях. Це дозволяє серверу не блокувати робочі потоки під час звернень до бази даних, забезпечуючи швидку відповідь клієнту навіть при великій кількості одночасних запитів до каталогу. Другий атрибут надійність (Reliability) гарантується строгими перевітками на сервісному рівні. Під час процедури бронювання система перевіряє наявність вільних примірників. Якщо параметр вільних копій менший або дорівнює нулю, сервісний шар генерує виняток і блокує транзакцію, що унеможлиблює видачу неіснуючих книг та виникнення від'ємних залишків у фізичному фонді. Аналогічний підхід з перевіркою залишку та генерацією винятків був продемонстрований у прикладі на Java для банківського рахунку. Третій атрибут безпека (Security) імплементовано через використання DTO замість прямих моделей бази даних. Внутрішні моделі ніколи не повертаються безпосередньо клієнту, що надійно приховує структуру таблиць від кінцевого користувача та захищає систему від атак типу перевизначення даних. Четвертий атрибут ремонтпридатність (Maintainability) забезпечується механізмом впровадження залежностей (Dependency Injection). Усі класи взаємодіють між собою виключно через інтерфейси, що дозволяє легко масштабувати код або змінювати реалізацію бази даних без необхідності переробляти бізнес-логіку.

У проєкті було успішно імплементовано п'ять основних бізнес-сценаріїв. Перший сценарій перегляду каталогу реалізує отримання списку доступних книг для читачів із можливістю фільтрації за авторами та статусом відцифрування. Другий сценарій додавання книги дозволяє бібліотекарю реєструвати нові надходження до фонду із зазначенням загальної кількості примірників та видавництва. Третій сценарій реєстрації читача відповідає за

створення профілю нового користувача в системі. Четвертий сценарій бронювання книги забезпечує створення ордеру із закріпленням фізичного екземпляра за читачем та автоматичним зменшенням лічильника доступних копій. П'ятий сценарій повернення книги відповідає за закриття активного ордеру з оновленням статусу та автоматичним збільшенням кількості вільних копій у каталозі бази даних.

Запити виконані в ході тестування:

Вихідні коди програми та історія комітів доступні у віддаленому репозиторії за посиланням <https://github.com/IvanTukalo/PTAPZ>

POST <https://localhost:7257/api/books>

Тіло запиту

```
{
  "title": "Патерни проектування",
  "authors": ["Еріх Гамма", "Річард Хелм"],
  "publisher": "Пітер",
  "year": 2020,
  "free_copies": 3,
  "is_digitized": false
}
```

The screenshot displays a REST client interface with the following sections:

- Responses:** A table showing a single response with status code 200 and description 'OK'.
- Curl:** A code block containing the curl command for the POST request.
- Request URL:** <https://localhost:7257/api/books>
- Server response:** A section with tabs for 'Code' and 'Details'.
- Code:** Shows the status code '201'.
- Details:** Shows the response body as a JSON object:

```
{
  "id": 1,
  "title": "Патерни проектування",
  "authors": [
    "Еріх Гамма",
    "Річард Хелм"
  ],
  "publisher": "Пітер",
  "year": 2020,
  "free_copies": 3,
  "booked_copies": 0,
  "is_digitized": false
}
```
- Response headers:**

```
content-type: application/json; charset=utf-8
date: Wed, 06 May 2020 07:03:11 GMT
server: Kestrel
```

POST <https://localhost:7257/api/users>

Тіло запиту

```
{
  "full_name": "Олександр Петренко",
  "phone": "+380671112233",
  "verification_code": "7788"
}
```

Curl

```
curl -X 'POST' \
  'https://localhost:7257/api/users' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "full_name": "Олександр Петренко",
    "phone": "+380671112233",
    "verification_code": "7788"
  }'
```

Request URL

https://localhost:7257/api/users

Server response

Code	Details
201	Response body <pre>{ "id": 1, "full_name": "Олександр Петренко", "phone": "+380671112233", "password": "123" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 06 May 2026 07:04:03 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

GET https://localhost:7257/api/books
Author "Xe"

Curl

```
curl -X 'GET' \
  'https://localhost:7257/api/books?author=3D0FA53D0XB5' \
  -H 'accept: */*'
```

Request URL

https://localhost:7257/api/books?author=3D0FA53D0XB5

Server response

Code	Details
200	Response body <pre>{ { "id": 1, "title": "Патерни проектування", "authors": ["Ерік Гамач", "Річард Хелл"], "publisher": "Нітер", "year": 2020, "free_copies": 3, "booked_copies": 0, "is_digitized": false } }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 06 May 2026 07:05:02 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

POST https://localhost:7257/api/orders
Тіло запиту

```
{
  "user_id": 1,
  "book_id": 1,
  "duration_days": 14
}
```

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7257/api/orders' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": 1,
    "book_id": 1,
    "duration_days": 14
  }'
```

Request URL

https://localhost:7257/api/orders

Server response

Code	Details
201	Response body <pre>{ "id": 1, "user_id": 1, "book_id": 1, "issue_date": "2026-05-06T07:06:45.6926465Z", "return_date": "2026-05-20T07:06:45.6926746Z", "status": "booked" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 06 May 2026 07:06:45 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

PATCH https://localhost:7257/api/orders/1

Тіло запиту

```
{
  "status": "returned"
}
```

Responses

Curl

```
curl -X 'PATCH' \
  'https://localhost:7257/api/orders/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "status": "returned"
  }'
```

Request URL

https://localhost:7257/api/orders/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "user_id": 1, "book_id": 1, "issue_date": "2026-05-06T07:06:45.6926465Z", "return_date": "2026-05-20T07:06:45.6926746Z", "status": "returned" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 06 May 2026 07:08:08 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

Висновки: у ході виконання лабораторної роботи було досліджено та практично застосовано архітектурний патерн Layered Architecture для розробки REST API системи автоматизації роботи бібліотекаря. Чітке розділення системи на рівні представлення, бізнес-логіки та роботи з даними дозволило написати масштабований код та успішно впровадити атрибути якості щодо продуктивності, надійності, безпеки та ремонтпридатності. Успішна імплементація п'яти ключових сценаріїв підтвердила дієздатність обраної архітектури та забезпечила повну відповідність розробленого програмного інтерфейсу визначеним технічним вимогам.